

Lecture 03 — Conditionals — the program that chooses

Algorithmic Thinking · Week 2, Session 3 — Tuesday 29 September 2026,
17:35–18:55 (Quick exam 1 in the first 20 minutes)

Leopoldo Rodríguez Salazar · Tecnológico de Monterrey

Contents

Overview	1
Opener — Explore or exploit? (10 min)	1
Situación problema — five minutes, today, so it is not a surprise (5 min)	2
Notes	2
Stretch — if the session was easy	7
After the session	7

Objective. Write a program whose output depends on a condition, and predict which branch runs before running it.

Overview

Quick exam 1 takes the first twenty minutes. Then, for the first time, the machine gets to **choose**.

Everything so far ran top to bottom: read, compute, display. That is why the time = 0 note has been sitting in the margin since Tuesday of week 1 — a program that cannot choose cannot refuse a bad input; it can only crash on it. Today it stops crashing.

One idea: **a condition is an expression whose value is True or False, and if runs a block only when it is True**. Everything else — elif, else, and, or, not, nesting — is that idea, applied carefully.

Opener — Explore or exploit? (10 min)

Algorithms to Live By, chapter 2. Friday night in Querétaro: the restaurant you love, or the new one that opened last month? Every time you choose the known one you are **exploiting**; every time you try a new one you are **exploring**. Exploring costs a possibly-bad dinner; exploiting costs never finding something better.

Take three answers before showing anything. Someone will say “depends how long I’m staying in Querétaro” — that person has found the whole chapter: **the value of exploring depends on how much time you have left to use what you find**. New in town: explore. Last week before you leave: exploit. That is also why a hospital trial and a casino look different, and why Netflix’s front page is a compromise.

The oldest simple strategy — Robbins, 1952 — fits on one line:

Win-stay, lose-shift. If the last choice was good, repeat it. If it was bad, switch.

It is an algorithm. Finite (one decision per night), ordered, unambiguous once you define “good”, input (last outcome) and output (tonight’s choice). Better strategies exist — the Gittins index, “upper confidence bound”, *optimism in the face of uncertainty* — but every one of them has the same skeleton: **a condition, and two branches.**

Your turn, 3 minutes, on paper: write win-stay-lose-shift in our notation. INPUT: (what do you need to remember from last time?), OUTPUT: , numbered steps. Circle the step that is a *decision*. Then the pivot, out loud: *that circled step is what Python learns today.*

Situación problema — five minutes, today, so it is not a surprise (5 min)

The **situación problema** is 15 % of the grade and it is the one piece of the course that is *yours*: you pick a business problem and a real dataset, and by the end of November you have a Python program that loads it, describes it statistically, and plots the answers to questions you wrote yourself.

- **Teams of two** for the three deliverables; the **final delivery is individual.**
- **Deliverable 1 — Friday 6 November:** the dataset, as an Excel table: 30–50 rows, up to 10 columns, 5 text and 5 numeric, from a real source (Kaggle, datos.gob.mx, the World Bank, or a company that lets you use its data — a family business counts).
- **Deliverable 2 — Friday 13 November:** four to six *driving questions* about your data.
- **Deliverable 3 — Friday 20 November:** the data loaded into Python and printed.
- **Final — Friday 27 November, individual:** the program (functions for descriptive statistics, two plots, a menu, a main) and a short report — and **you explain your own code out loud.**
- Each missed or late stage costs 5 points off the project grade.

Nothing to do yet except one thing: **start noticing data.** The formal brief comes out on Friday 16 October; the full text is already on the course site under *Situación problema*. Everything we do between now and then — conditionals today, functions Friday, loops in weeks 3–4, lists in week 7 — is a piece of that program.

Notes

The bug we could not fix

Week 1, Tuesday, on the board:

```
-- what if time = 0? --
```

Friday’s program made it concrete:

```
distance_km = float(input("Distance in km: "))
hours = float(input("Time in hours: "))
speed = distance_km / hours          # ZeroDivisionError when hours is 0
print("Average speed:", speed, "km/h")
```

Type 0 for the time and Python stops: ZeroDivisionError: float division by zero. It is the third kind of error you have met. TypeError was text where a number was needed;

133.33 was a program that ran and lied; this one is a **correct program given an input it was never told how to handle**. The algorithm never said what to do when time is zero, so the program does not know either. 01_time_zero.py.

A condition has a value

Before if, the thing if looks at. Type these in the Thonny shell, one at a time:

```
>>> 5 > 3
True
>>> 5 == 3
False
>>> hours = 0
>>> hours == 0
True
>>> hours > 0
False
>>> type(5 > 3)
<class 'bool'>
```

True and False are values, of type bool — as much a value as 5 or "hola". A **relational expression** produces one:

Operator	Meaning	5 ? 3
<	less than	False
<=	less than or equal	False
>	greater than	True
>=	greater than or equal	True
==	equal — two signs	False
!=	not equal	True

Gloss: *expresión relacional* · *booleano* — boolean · *condición* — condition · *rama* — branch · *sangría* — indentation · *anidado* — nested

= is a command; == is a question. x = 5 puts 5 in the box. x == 5 asks whether the box holds 5, and the answer is True or False. Python catches if x = 5: as a syntax error, but not x == 5 on its own line, which does nothing and says nothing. 02_relational.py.

if — a block that runs only when the condition is True

```
hours = float(input("Time in hours: "))

if hours > 0:
    speed = distance_km / hours
    print("Average speed:", speed, "km/h")
```

Three things on the line if hours > 0: — the keyword, a condition, and a **colon**. Then the block: every line indented **four spaces** under it belongs to it. Un-indent, and you are back in the main program, which runs regardless. 03_if_block.py shows the difference: move print one level left and it runs even when there was no speed to print — NameError.

else — the other branch, and the end of the margin note

```
if hours > 0:
    speed = distance_km / hours
    print("Average speed:", speed, "km/h")
else:
    print("Time must be greater than zero.")
```

Exactly one of the two blocks runs. That is the `time = 0` note, closed: the algorithm now says what happens on a bad input, and the program says it too. `04_speed_checked.py`.

Write the algorithm for it before the code — it is one line longer than week 1's:

```
1. READ distance, time
2. IF time > 0:
3.     speed <- distance / time
4.     DISPLAY speed
5. ELSE:
6.     DISPLAY "time must be greater than zero"
```

elif — more than two branches, in order

A grade message: 0-69 fails, 70-79 good, 80-89 very good, 90-100 excellent, anything else is an error.

```
grade = float(input("Grade: "))
```

```
if grade < 0 or grade > 100:
    print("ERROR")
elif grade < 70:
    print("Reprobado")
elif grade < 80:
    print("Bueno")
elif grade < 90:
    print("Muy bueno")
else:
    print("Excelente")
```

Read it the way Python does: top to bottom, **the first True condition wins and the rest are skipped**. That is why `elif grade < 80` does not need to say `grade >= 70` — if we got there, the previous test already failed. And it is why **order matters**: swap the `< 70` and `< 80` lines and a 65 prints Bueno. `05_grade_message.py` has the working version and the swapped one; predict both for 65, 79, 80, 100, 101 before running.

and, or, not

A year is a leap year when it is divisible by 4 and not by 100, or when it is divisible by 400. That sentence is already the condition:

```
year = int(input("Year: "))
```

```
if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:
```

```

    print(year, "is a leap year")
else:
    print(year, "is not a leap year")

```

% is the remainder — `year % 4 == 0` means *divisible by 4* — and here is the first time it earns its place. Precedence: not first, then and, then or; the parentheses above are not required, but they make the sentence readable, which is the point. Tests: 1984 yes, 1800 no, 2000 yes, 2026 no. 06_leap_year.py.

Nested if — a choice inside a choice

Sometimes the second question only makes sense after the first has been answered:

```

n = int(input("Integer: "))

if n > 0:
    if n % 2 == 0:
        print("positive even")
    else:
        print("positive odd")
else:
    if n % 2 == 0:
        print("negative even")
    else:
        print("negative odd")

```

Eight lines of indentation doing the work of four sentences. It is correct — and it is worth seeing next to the flat version with `and`, which says the same thing in four `elif` rungs. Neither is “right”; the nested one mirrors *the order you would ask a person*, the flat one is easier to test. What matters is that `0` goes somewhere on purpose — in this version it is “negative even”, which is a bug the algorithm should have caught. 07_nested.py.

Two more shapes: the conditional expression, and `match`

Both say something you can already say with `if`. Neither is required; both are worth recognising, because you will read them in other people’s code.

The conditional expression (other languages call it the *ternary operator*) — an `if` that *is a value*:

```

label = "leap year" if is_leap else "not a leap year"
print(year, "is a", label)

```

Read it left to right as English: *label is “leap year” if is_leap, else “not a leap year”*. Use it when the two branches are two **values** for the same variable and nothing else happens. The moment a branch needs two lines, go back to `if`. 08_ternary.py.

match / case (Python 3.10 and later) — the switch of other languages, for the case where one value is compared against several **specific** alternatives:

```

option = int(input("1 stats · 2 chart 1 · 3 chart 2 · 4 exit: "))

```

```

match option:

```

```

case 1:
    print("Descriptive statistics")
case 2:
    print("Chart 1")
case 3:
    print("Chart 2")
case 4:
    print("Exit")
case _:
    print("Not an option")

```

case `_`: is the *else*. This is the menu of the situación problema's final program, and it reads better than four `elif option == rungs`. It does **not** replace the ladder for *ranges* (`grade < 70`): match compares against values, not conditions (it can, with `case x if x < 70`; but at that point the ladder is clearer). `09_match.py`.

Exam note. Trinket's Python may be older than 3.10 — match may not exist in the quick-exam editor. The `if / elif` ladder always works. Know match; do not depend on it.

Failure-first: three bugs that do not crash

1. `= for ==` inside an expression is a syntax error — fine, Python helps. But if `hours = 0`: written as two lines `hours = 0` then `if hours:` silently tests *whether hours is non-zero*. Bugs that run are the ones to fear.
2. **A boundary on the wrong side.** `grade < 70` vs `grade <= 70`: a 70 fails or passes depending on one character. **Test the boundaries** — 69, 70, 71 — not the middle.
3. **The unreachable branch.** `if x > 10: ... elif x > 20: ...` — the second can never run. No error, no warning; the branch is simply dead. Order the conditions from most to least specific, or from smallest to largest, and *say which* in the algorithm.

In-class activity — Speed, checked and classified (*individual, 20 min, graded — the first of the four in-class activities, 4 % of the course*)

Finish the week-1 program properly.

1. Read distance (km) and time (hours) from the user.
2. If the time is not greater than zero, display Time must be greater than zero. and nothing else.
3. Otherwise compute the average speed and display it, then classify it:
 - below 40 km/h → slow
 - 40 to below 90 km/h → normal
 - 90 to 120 km/h → fast
 - above 120 km/h → too fast — this is a road, not a runway

Algorithm first, on paper — INPUT:, OUTPUT:, numbered steps with the IF lines. Then the program, in the exam format (name in a `#` comment on line 1, algorithm in `"""`). Then the tests — write them under the algorithm:

distance	time	expected
100	0	Time must be greater than zero.

distance	time	expected
100	2	50.0 → normal
100	1	100.0 → fast
120	1	120.0 → fast
121	1	121.0 → too fast
30	1	30.0 → slow

The two 120/121 rows are there on purpose: the boundary is where the marks are.

Submit: photo of the algorithm + A_Matricula_ic1.py → Canvas → *In-class activity: Conditional structure (if)* — by Sunday 23:59, but the work happens now, in the room.

Rubric: algorithm with input/output and every branch stated 40 % · program runs and passes all six tests 40 % · boundaries right (120 → fast, 121 → too fast, 0 → error) 20 %.

Stretch — if the session was easy

Optional, ungraded, and harder than they look. Bring them to asesorías or leave them in the week's evidence box.

1. **No if.** Write `speed_label(speed)` (slow / normal / fast / too fast, same thresholds as the in-class activity) **without** `if`, `elif`, `else` or the conditional expression. Hint: `True + True` is 2. Then say, in one sentence, why the `if` version is still the one you should hand in.
2. **The specification hunt.** Take the client-status table from Friday's lab (*más de \$1,000,000 → Excelente, ...*). List every input the table does **not** decide: the exact limits, negatives, decimals, text. That list is the algorithm's missing lines; write them.
3. **Prove the leap year.** Write the condition as one boolean expression *and* as nested ifs, and show they agree for every year from 1 to 2400. You may use anything you already know; the point is the word *every*.

After the session

Homework 2 — assignments/hw-02-conditionals.md. Due Sunday 4 October, 23:59, on Canvas. Two programs: leap year (and/or/%) and three numbers in descending order (nested comparisons). Algorithm on paper, exam file format, tests written under the algorithm.

Also before Friday:

1. **Levelling module** — lessons 6–8 (flowcharts) by Friday; all ten by Thursday 8 October.
2. **Lab 2 — Friday 16:30**, with Luis: conditionals, the same lab as every group (*Laboratorio: Estructura condicional if*). Bring the laptop.
3. **Friday's session: functions.** Read *Self-study: Python functions* in Canvas — ten minutes — so that `def` is not the first time you see the word.
4. **Quick exam 2 is Tuesday 6 October**, first 20 minutes: conditionals and functions.